



دانشکده مهندسی کامپیوتر

سیستم‌های عامل

فاز ۱ پروژه

نیم‌سال اول

سال تحصیلی ۱۴۰۴-۱۴۰۵

در انجام پروژه به نکات زیر توجه نمایید:

- برای تحویل پاسخ این پروژه لازم است در یک فایل zip. تمام فایل هایی که تغییر دادید + یک گزارش مختصر از آنچه انجام داده‌اید را قرار بدید و یک فایل به نام OS_P1_student-id.zip در سامانه کورسز ارسال کنید (جزئیات بیشتر در بخش **توضیحات پایانی** توضیح داده شده است).
- کامنت گذاری در کد جهت افزایش خوانایی انجام شود و هر تکه از کد در بخش مخصوص خود نوشته شود.
- در صورت هر گونه مشکل می‌توانید به اکانت‌های تلگرام زیر پیام دهید:
@Parsa22000 و @lkavehamdl
- در صورت کشف تقلب نمره ۰ برای تمام طرف‌های درگیر در نظر گرفته خواهد شد.
- مهلت انجام این پروژه تا ساعت ۲۳:۵۹ **جمعه ۱۶ آبان** است.

فهرست مطالب

۳	 مقدمه
۴	 نصب XV6
۴	 نصب با استفاده از docker
۴	 نصب در سیستم linux (debian based)
۵	 اجرای XV6
۶	 پیش از شروع
۶	 اضافه کردن system call به XV6
۸	 بخش اول: System Call Counter
۹	 بخش دوم: Process Tree
۱۱	 بخش سوم: Reparenting
۱۳	 توضیحات پایانی
۱۳	 توضیحات
۱۳	 آنچه که باید ارسال کنید

مقدمه

همانطور که در کلاس درس بیان شد پروژه درس سیستمهای عامل مربوط به شناخت کامل سیستم عامل آموزشی XV6 و افزودن قابلیت‌های جدید به آن است. با انجام دقیق این پروژه و تطبیق مفاهیمی که در کلاس درس معرفی شده است به یادگیری عمیق و دائمی شما کمک خواهد کرد. برنامه نویسی در سطح سیستم عامل به شما کمک می‌کند تا تجربه ای منحصر به فرد داشته باشید و دید بهتری نسبت به آنچه که در زبانهای سطح بالاتر می‌نویسید بدست آورید.

هدف از دو فاز ابتدایی این پروژه آشنایی شما دانشجویان با این سیستم عامل آزمایشی است و روند کار به این صورت است که در فاز اول به صورت فردی و در فاز دوم و سوم به صورت گروهی باید تغییراتی در این سیستم عامل ایجاد کنید تا بتوانید درک خوبی از این سیستم عامل را بدست آورید. در فاز اول پروژه بایستی درک مناسبی از برخی مفاهیم پایه ای از جمله پردازنده‌ها، سیستم کال‌ها، و عملکرد کلی سیستم عامل را پیدا کنید و در نهایت دو سیستم کال ساده را به سیستم عامل خود اضافه کنید.

در این ترم ما با آخرین نسخه از سیستم عامل XV6 که بر اساس معماری پردازنده RISC-V توسعه یافته است کار خواهیم کرد. برای آشنایی با این سیستم عامل میتوانید از لینک زیر استفاده کنید

<https://pdos.csail.mit.edu/6.828/2022/xv6.html>

برای اجرای این سیستم عامل می‌توانید مراحل و پیش‌نیاز های build کرن آن را از لینک زیر دریافت کنید:

<https://pdos.csail.mit.edu/6.828/2022/tools.html>

توصیه ما برای MAC OS استفاده از Docker Container است. به این صورت که image محیط اجرایی XV6 در غالب یک container اجرا می‌کنیم. در ادامه به نحوه اجرای آن می‌پردازیم.

برای سیستم های linux پیشنهاد ما نصب مستقیم XV6 است که در ادامه شیوه نصب آن گفته می‌شود
برای سیستم های ویندوز هر کدام از دو روش docker یا استفاده از ماشین مجازی لینوکس قابل قبول است.

نصب XV6

نصب با استفاده از docker

ابتدا با استفاده از لینک زیر نرم افزار docker desktop را نصب کنید.

<https://docs.docker.com/get-docker/>

سپس برنامه Docker Desktop را اجرا و از فعال شدن Docker Engine داخل آن اطمینان حاصل نمایید. اکنون باید image را دریافت کنید که با استفاده از دستور زیر، می‌توانید کانتینر watkuo/XV6-env را دریافت کنید:

```
$ wtakuo/XV6-env pull docker
```

کد مربوطه را در پوشه دلخواه (فرضا `XV6-riscv`) ذخیره کنید. داخل این پوشه دستور زیر را اجرا کنید:

```
$ docker run -it --rm -v \$(pwd):/home/XV6/XV6-riscv wtakuo/XV6-env
```

چنان چه در ویندوز کار می‌کنید از دستور زیر استفاده کنید:

```
$ docker run -it --rm -v "%cd%:/home/XV6/XV6-riscv wtakuo/XV6-env"
```

نصب در سیستم linux (debian based)

ابتدا با استفاده از دستور زیر dependency های مورد نیاز را نصب کنید:

```
$ sudo apt install build-essential qemu-system-misc git
```

برای چک کردن نصب qemu می‌توانید از دستور زیر استفاده کنید:

```
$ qemu-system-riscv64 --version
```

بعد نیاز دارید فایل xv6 را دانلود کنید؛ در یک دایرکتوری دلخواه دستور زیر را اجرا کنید:

```
$ git clone https://github.com/mit-pdos/XV6-riscv.git
```

در مرحله بعد نیاز به build کردن پروژه داریم که این کار را با استفاده از cross compiler RISC-V انجام می‌دهیم. برای نصب آن از دستور زیر استفاده کنید:

```
$ sudo apt install gcc-riscv64-unknown-elf
```

اجرای XV6

بعد از رفتن به پوشه‌ای که XV6 در آن قرار دارد (در این مثال پوشه xv6-riscv)، با دستورهای زیر می‌توانید کار خود را با سیستم‌عامل شروع کنید:

- `make`: این دستور پروژه را کامپایل و build می‌کند و پروژه آماده اجرا می‌شود.
- `make clean`: با این دستور تمام فایل‌های تولید شده از build قبلی (مانند فایل‌های `s.o` و `o` تولید شده^۱) را پاک می‌کند. این کار باعث می‌شود در build بعدی از فایل‌های کامپایل شده استفاده نشود و فایل‌ها دوباره از نو کامپایل شوند.
- `make qemu`: در صورت نیاز پروژه را build می‌کند (اگر از قبل `make` را اجرا کرده باشید از این مرحله عبور می‌کند) و سیستم‌عامل را با شبیه‌ساز `qemu`^۲ اجرا می‌کند.

بعد از هر تغییر در کد، برای تست سیستم‌عامل، ابتدا `make clean` و سپس `make qemu` را اجرا کنید.

^۱ به غیر از فایل‌های `s.o` خود پروژه

^۲ شبیه‌ساز `qemu` یک virtual machine برای تست‌های سطح پایین سیستم‌عامل است.

پیش از شروع

پیش از شروع خوب است که اطلاعات کافی راجع به نحوه اجرا شدن سیستم عامل آزمایشیمان و نحوه اجرا شدن اولین پرده اطلاعات کافی بدست آوریم. همچنین می‌توانید از کتاب `xv6: a simple, Unix-like teaching operating system` به عنوان منبع استفاده کنید.

اضافه کردن system call به XV6

در فاز اول پروژه می‌خواهیم که ۲ سیستم کال جدید به این سیستم عامل آموزشی اضافه کنیم. خوب است که قبل از هرچیز مروری بر مراحل اضافه کردن سیستم کال داشته باشیم

I- در صورت نیاز اضافه کردن struct جدید در هر فایل مربوطه

II- در صورت تعریف یک تایپ جدید یا یک تابع جدید که قرار است در فایل های دیگر مورد استفاده قرار گیرد، تعریف آن را در فایل `defs.h` مانند نمونه های قبلی قرار دهید. در کل هر تایپ یا تابعی که در فایل دیگری تعریف شده و لازم است در فایل های دیگر مورد استفاده قرار گیرد باید پروتوتایپ آن در این فایل قرار بگیرد.

III- در فایل `syscall.h` سیستم کال جدید را با یک شماره ی جدید اضافه کنید.

IV- در فایل `syscall.c` نام تابع سیستم کال مورد نظر را به نامی که در فایل `syscall.h` قرار دادید مپ کنید.

V- پروتوتایپ تابع مورد نظر را نیز در ابتدای فایل `syscall.c` اضافه کنید.
(extern "function" definition)

VI- پیاده سازی تابع سیستم کال مورد نظر را با نامی که در `sysproc.c` قرار دهید. در این بخش اصولاً بخش انتقال و تبدیل پارامترها انجام می شود و در فایل `proc.c` پیاده سازی اصلی سیستم کال انجام می شود.

VII- برای اینکه برنامه‌های کاربر بتوانند از این سیستم کال‌ها استفاده کنند لازم است در فایل `usys.pl` سیستم کال مربوطه را اضافه کنید.

VIII- پروتوتایپ تابع مورد نظر را در فایل `user.h` اضافه کنید.

IX- در نهایت `Makefile` را تغییر دهید و اسم فایل برنامه‌ی کاربری که نوشتید را در بخش `UPROGS` اضافه کنید.

بخش اول: System Call Counter

در این بخش قرار است یک سیستم کال جدید به اسم `sysclcnt` به سیستم عامل آموزشی XV6 اضافه کنید.

این `system call` ورودی نداشته و در خروجی باید یک عدد برگرداند که نشان دهنده این است که از زمان `boot` شدن این سیستم عامل تا الان چند مرتبه سیستم کال صدا زده شده است.

```
1 int sysclcnt();
```

به عنوان مثال فرض کنید در بار اول صدا زدن سیستم کال `sysclcnt` مقدار n برگردانده شود. در صورتی که یک بار سیستم کال `read` را صدا بزنید و سپس مجدد سیستم کال `sysclcnt` را صدا بزنید، مقدار $n + 2$ باید در خروجی برگردانده شود چرا که `sysclcnt` خود نیز به عنوان یک سیستم کال شمرده می شود.

بخش دوم: Process Tree

در این بخش قرار است یکی سیستم کال جدید به اسم ptree به سیستم عامل آموزشی XV6 اضافه کنید.

در سیستم عامل‌ها، هر process توسط یک process دیگر ساخته می‌شود که این امر باعث ایجاد ساختار درختی می‌شود.

در این بخش شما باید سیستم کال ptree را به شکل زیر پیاده سازی کنید:

```
1 int ptree(int PID, struct proc_tree* proc_tree);
```

به طور کلی چنین سیستم کالی خواهیم داشت که ورودی آن یک عدد است که نشان دهنده PID مربوط به root درخت ما است و یک پوینتر به struct که آن را پر کند و در صورت موفقیت باید خروجی ۰ برگرداند.

برای proc_tree چنین ساختاری پیشنهاد می‌شود (در صورت نیاز می‌توانید آن را تغییر دید):

```
1 struct proc_info
2 {
3     char name[16];
4     int pid;
5     int child_count;
6     struct proc_info* children[NPROC];
7 }
```

```
1 struct proc_tree
2 {
3     int count;
4     struct proc_info procs[NPROC];
5     struct proc_info root;
6 }
```

برای چاپ کردن درخت به صورت بازگشتی می‌تونید از شبه کد الگوریتم زیر کمک بگیرید (کاراکترهای "└"، "├"، و "│" را از همینجا کپی کنید):

Algorithm 1 Print Process Tree

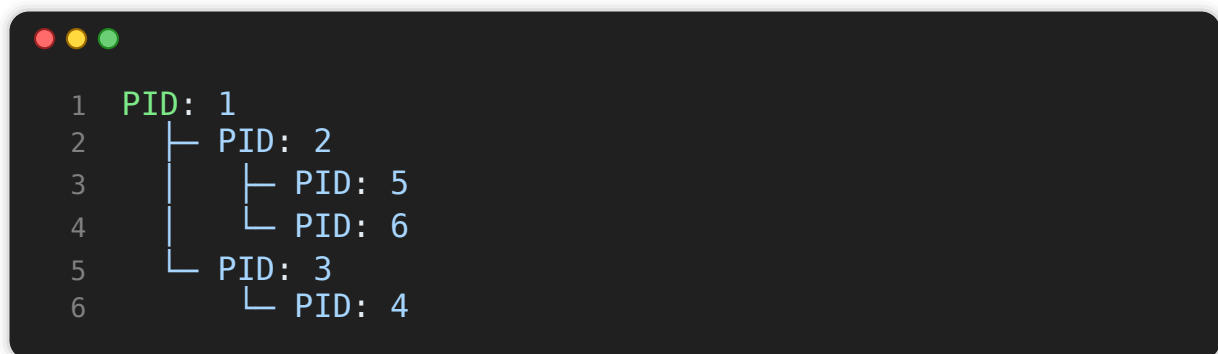
```

1: function PRINT-PROCESS-TREE(root)
2:   return PRINT-PROCESS-TREE-RECURSIVE(root, "", True)

3: function PRINT-PROCESS-TREE-RECURSIVE(proc, prefix, isLast)
4:   if prefix == "" then
5:     print(proc.PID)                                ▷ root process
6:   else if isLast then
7:     print(prefix + "└" + proc.PID)
8:   else
9:     print(prefix + "├" + proc.PID)
10:  count ← length(proc.children)
11:  for i = 0 to count - 1 do
12:    child ← proc.children[i]
13:    if isLast then
14:      newPrefix ← prefix + "\t"                    ▷ you can use double spaces instead
15:    else
16:      newPrefix ← prefix + "│ "                    ▷ be aware of the space near "│"
17:    PRINT-PROCESS-TREE-RECURSIVE(child, newPrefix, i equals count - 1)

```

در نهایت خروجی کد شما باید مانند نمونه زیر باشد:



```

1  PID: 1
2  │ PID: 2
3  │ │ PID: 5
4  │ │ PID: 6
5  │ PID: 3
6  │ │ PID: 4

```

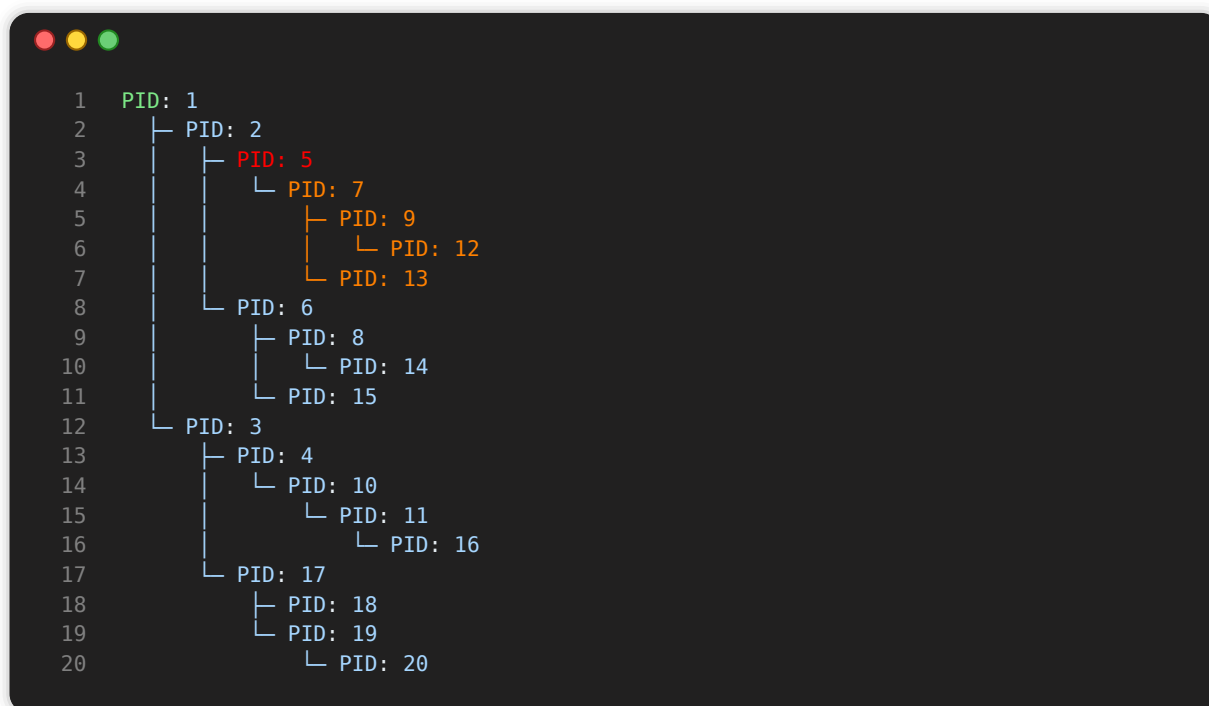
بخش سوم: Reparenting

برای این بخش باید ابتدا سیاست سیستم عامل آموزشی XV6 در هنگام تولید پردازش‌های orphan را بررسی کنیم:

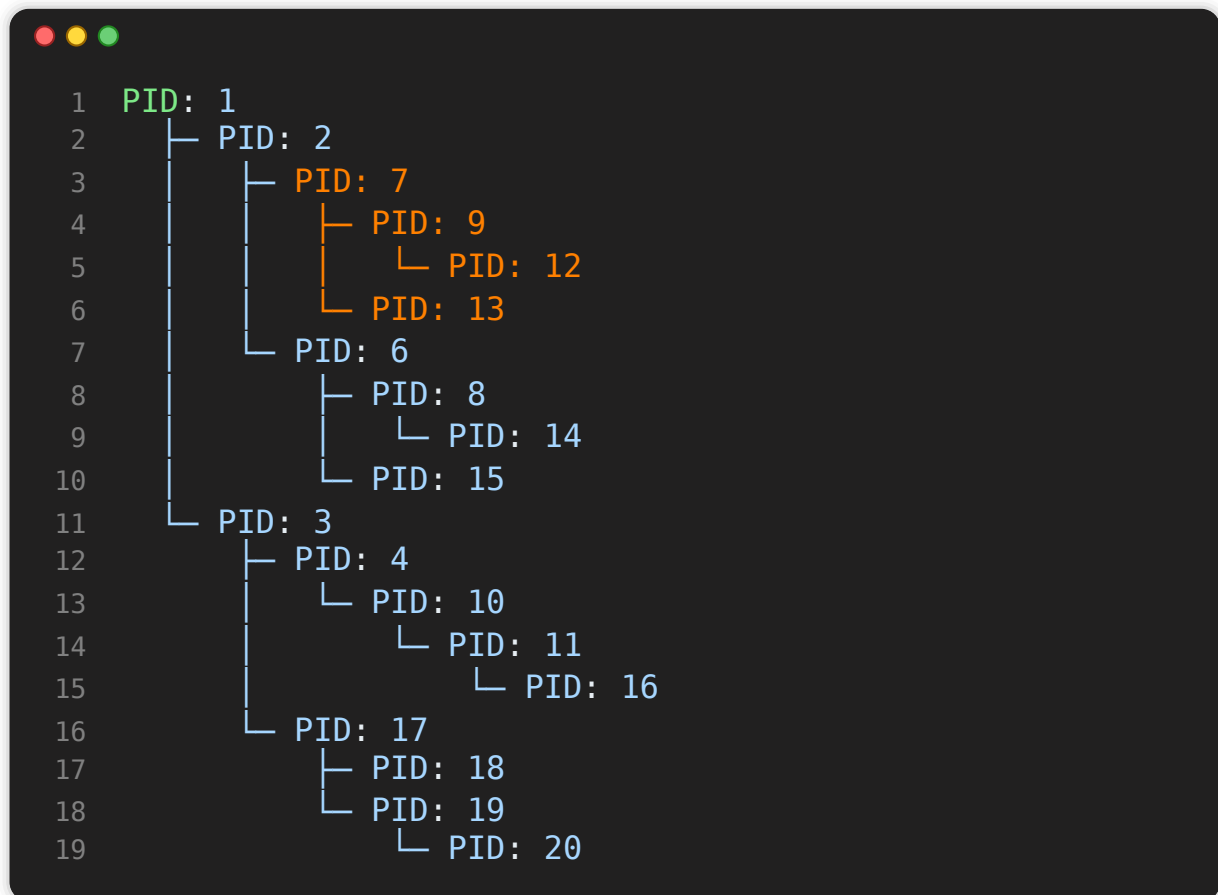
در سیستم عامل های Linux (POSIX-like) زمانی که یک process از بین می‌رود، فرزندان آن کشته نمی‌شوند. پردازش های init یا systemd (بسته به نوع سیستم عامل) این process را adopt می‌کند.

در سیستم عامل آموزشی XV6 نیز همین ایده را داریم. و برای این بخش شما باید تغییراتی در کد این سیستم عامل ایجاد کنید که بعد از از بین رفتن یک process، فرزندان آن بجای init توسط والد process از بین رفته adopt شوند.

به عنوان مثال درخت زیر را در نظر داشته باشید:



در صورتی که پردازش PID:5 از بین برود، بجای این که پردازش PID:7 و نوادگانش به PID:1 (که همان init است) reparent شوند، به پردازش PID:2 reparent می‌شوند:



توضیحات پایانی

از شما درخواست داریم که یک private repository در گیت‌هاب درست کنید و تغییرات کد خود را مرحله به مرحله commit کنید و در صورت تمایل می‌توانید هر یک از تدریس‌یاران را به پروژه خود اضافه کنید. دقت کنید که شما نبایستی برنامه‌های خود را با دیگر دانشجویان به اشتراک بگذارید.

آیدی گیت‌هاب ما:

کاوه: lkavehamdl

پارسا: Parsa2201

توضیحات

- این فاز پیش‌نیاز قطعی فازهای بعدی است و انجام ندادن آن باعث می‌شود که نتوانید فاز دوم را شروع کنید. و همچنین نمی‌توانید برای انجام پروژه گروهی را تشکیل دهید.
- پروژه شما تحویل آنلاین خواهد داشت بنابراین از استفاده از کدهای دیگران یا کدهای موجود در وب که قادر به توضیح دادن عملکرد آنها نیستید، بپرهیزید.
- ابهامات خود را در گروه درس در تلگرام مطرح کنید و ما در سریع‌ترین زمان ممکن به آنها پاسخ خواهیم داد.

آنچه که باید ارسال کنید

یک فایل zip با نام OS_P1_student-id.zip (student-id را با شماره دانشجویی خود جایگزین کنید) که شامل دو مورد زیر است:

- گزارش خیلی مختصر از آنچه که انجام داده‌اید تا دو فراخوانی سیستمی خواسته شده را به xv6 اضافه کنید.
- پوشه‌ای که در آن کدهای شما وجود دارد. دقت کنید که تنها و تنها فایل‌هایی را که تغییر داده‌اید یا اضافه کرده‌اید را برای ما بفرستید.